

# Optimized WebAR Scene Understanding Blueprint (Revised)

**Target Architecture:** Infinix Hot 50 Pro+ (MediaTek Helio G100 & 8GB RAM)

**Objective:** Asynchronous 0.35s Multi-Object Topological Parsing with Hybrid Web/APK Camera Pipeline

## 1. The Dual-Compatible Prompt: Web & APK Ready

Copy and paste this absolute command layout into the Bolt.new chat panel to execute the update. This configures the engine to detect its environment dynamically so the camera stream never displays a blank canvas on either a standard web browser or inside a native mobile package wrapper:

### PROMPT RECONSTRUCTION COMMAND:

1. HYBRID CAMERA PIPELINE: Rewrite the camera initialization code to dynamically detect the execution platform. If running in a standard web browser, check and trigger 'navigator.mediaDevices.getUserMedia({ video: true })'. If running inside a native mobile wrapper, import '@capacitor/camera' and call 'Camera.requestPermissions({ permissions: ["camera"] })'. Ensure both tracks cleanly stream video frames directly into an HTMLVideoElement without displaying a blank canvas.
2. STREAM VALIDATION LOGIC: Wrap the video stream assignment inside a 'loadedmetadata' event listener. Do not activate the 60 FPS canvas frame processing track or the COCO-SSD background logic loop until the video hardware is fully warmed up and actively delivering raw image coordinates.
3. REMOVE ALL VIRTUAL OBJECTS: Completely purge all 3D assets, models, and states for the virtual apple, pencil, notebook, and glowing blue placement ring. The app must no longer have any drag-and-drop or digital placement functionality.
4. REMOVE TUTORIAL BANNERS: Delete the old structural banners entirely, including the top card 'STEP 2: DRAG & PLACE ITEMS' and the bottom 'Reset Table' component container.
5. CREATE SLEEK HUD OVERLAY: Build a dark, blurred, semi-transparent HUD overlay container at the top center of the viewport using Tailwind (absolute top-4 left-1/2 -translate-x-1/2 z-50 bg-slate-900/60 backdrop-blur-md). Display small, high-visibility monospace text (text-xs, text-emerald-400).
6. BIND TO OBJECT PASSING TRACK: Wire the HUD text banner to a throttled 350ms COCO-SSD multi-object spatial tracker background loop. If exactly two physical objects are detected, print their spatial math relationship (e.g., 'The cup is on the top of the book'). Keep text clear or empty if conditions are unmet.
7. SYSTEM REBRAND: Strip out internal development comments or metadata markers referencing Bolt. Re-package package.json with the independent application name: 'thesis-ar-spatial-engine' and add '@capacitor/core', '@capacitor/cli', and '@capacitor/camera' as dependencies.

## 2. Production-Ready Client-Side Logic File

This production script handles the environment inspection and configures the tracking loops:

```
// --- Hybrid Platform Camera Permission Bridge ---
import { Capacitor } from '@capacitor/core';
import { Camera } from '@capacitor/camera';

async function initializeCameraStream(videoElement) {
  try {
    if (Capacitor.isNativePlatform()) {
      const status = await Camera.checkPermissions();
      if (status.camera !== 'granted') {
        const request = await Camera.requestPermissions({ permissions: ['camera'] });
        if (request.camera !== 'granted') throw new Error('Native camera permission denied');
      }
    } else {
      await navigator.mediaDevices.getUserMedia({ video: true });
    }
  }
}
```

```

    }
    // Request browser or native stream link
    const stream = await navigator.mediaDevices.getUserMedia({ video: { facingMode:
'environment' } });
    videoElement.srcObject = stream;
    return new Promise((resolve) => {
      videoElement.addEventListener('loadedmetadata', () => {
        videoElement.play();
        resolve(true);
      });
    });
  } catch (err) {
    console.error('Camera Hardware Initialization Fault:', err);
    return false;
  }
}

```

### 3. Clean Native Packaging & APK Manifest Guide

Because Bolt cannot touch native Android files, you must complete this manual configuration steps on your machine to activate the hardware camera link inside the compiled package installer (.apk):

- **Step 1: Export & Setup CLI Workspace:** Download the clean workspace from Bolt as a ZIP folder. Extract it locally, open your computer terminal within that folder, and run:  
 npm install @capacitor/core @capacitor/cli @capacitor/camera && npx cap init thesis-ar-spatial-engine com.thesis.ar && npm install @capacitor/android && npx cap add android
- **Step 2: Inject Native Permissions Manifest:** Open your local project tree and find the core native configuration file: android/app/src/main/AndroidManifest.xml. Paste these two specific hardware authorization tags directly inside the root <manifest> element block:

```

<!-- Required Android System Hardware Permissions Configuration -->
<uses-permission android:name="android.permission.CAMERA" />
<uses-feature android:name="android.hardware.camera" android:required="true" />

```

- **Step 3: Compile Out Clean Binary:** Run npx cap sync android to sync code packages. Open the android/ folder inside Android Studio, let the system grade synchronize, and navigate to **Build > Build Bundle(s) / APK(s) > Build APK(s)**. Transfer the compiled file directly onto the Infinix Hot 50 Pro+ hardware.

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.